

TEMA 4. BUENAS PRÁCTICAS Y REFINAMIENTO



EN ESTE TEMA SERÁS CAPAZ DE...

1. Perfeccionar los resultados de una Inteligencia Artificial mediante técnicas de ajuste continuo, adaptando las respuestas a las limitaciones y necesidades técnicas del entorno local.
2. Descomponer problemas complejos de soporte, redes y bases de datos en pasos simples y ordenados, asegurando que cada instrucción de código o configuración sea precisa.
3. Validar de manera crítica y autónoma la veracidad del código y los datos generados por una IA, aplicando métodos de verificación cruzada con manuales oficiales para evitar fallas informáticas en los negocios y entornos institucionales de nuestra comunidad.



PRÁCTICA



Pregunta detonante: ¿Qué harías si estás configurando una red o una base de datos para un negocio local, le pides ayuda a una Inteligencia Artificial y el primer código o solución que te devuelve contiene errores graves que paralizan todo el sistema del cliente? ¿Te darías por vencido o sabrías cómo guiar a la IA para corregirlo?

El desafío del Técnico en Pailón: Imagina que trabajas como Técnico en Sistemas Informáticos y te han contratado para solucionar un problema urgente en un negocio comercial del **Barrio Central Fátima** (frente a la plaza principal y la iglesia). Este negocio necesita conectar su sistema de inventario local con un almacén ubicado en el **Barrio 24 de Septiembre** (cerca de la estación de tren). La conexión a internet en la zona a veces presenta alta latencia, por lo que requieres configurar un script de red optimizado en Linux y un esquema de base de datos en PostgreSQL que soporte reconexiones automáticas sin duplicar datos.

Abres una herramienta de Inteligencia Artificial y escribes un prompt rápido: "*Dame un código para conectar dos oficinas con PostgreSQL y Linux*". La IA te devuelve un script genérico con configuraciones para servidores en la nube de alta velocidad, comandos desactualizados y contraseñas por defecto visibles. Al probar el script en las computadoras del negocio en el Barrio Central Fátima, el sistema colapsa por completo, bloquea los puertos de red y el dueño del negocio se desespera porque está perdiendo ventas en ese mismo instante.

No tienes tiempo de reescribir todo el código desde cero de forma manual. Necesitas utilizar la misma IA, pero esta vez aplicando buenas prácticas para guiarla de forma progresiva, dividiendo el problema en partes pequeñas, verificando cada comando y ajustando los parámetros de red a la velocidad real de las conexiones de Pailón.



TEORÍA



4.1. TÉCNICAS DE ITERACIÓN Y AJUSTE PROGRESIVO DEL PROMPT

La interacción con una Inteligencia Artificial no es un proceso de un solo intento. En el trabajo cotidiano de un Técnico, la primera respuesta de un modelo de lenguaje suele ser general o teórica. La **iteración** es el proceso cíclico de refinar, corregir y pulir las instrucciones (prompts) que le damos a la IA basándonos en las respuestas previas que esta nos proporciona.

Para realizar un ajuste progresivo efectivo, se deben seguir tres pasos clave:

1. **Analizar la salida inicial:** Identificar qué partes del código o de la explicación sirven y cuáles no se adaptan a la realidad del problema.
2. **Inyectar restricciones locales:** Informar a la IA sobre las limitaciones del hardware, del sistema operativo exacto o del ancho de banda disponible.
3. **Solicitar modificaciones específicas:** En lugar de pedirle a la IA que "haga todo de nuevo", se le instruye para que modifique únicamente la línea de código o el parámetro erróneo.





Si el Técnico aplica esta técnica en el caso del Barrio Central Fátima, el segundo prompt de ajuste no sería genérico, sino específico: *"El script anterior falló porque usó comandos de una versión antigua de Ubuntu. Modifica el script de red para que funcione en Ubuntu Server 22.04 LTS, cambia el puerto por defecto de PostgreSQL al 5432 y añade un tiempo de espera (timeout) de 10 segundos debido a la baja velocidad de la conexión local"*.

4.2. DIVISIÓN DE TAREAS COMPLEJAS EN SUB-TAREAS SIMPLES (ENCADENAMIENTO)

Cuando un Técnico se enfrenta a un proyecto grande, como desarrollar un sistema de control de partidos y reservas para el coliseo municipal en el **Barrio 27 de Mayo**, pedirle a la IA todo el proyecto de golpe (*"Hazme un software completo para gestionar el coliseo"*) es un error grave. La IA omitirá detalles de seguridad, cometerá errores de lógica y entregará fragmentos de código incompletos.

El **encadenamiento de prompts (Prompt Chaining)** consiste en desglosar un problema técnico complejo en unidades de trabajo independientes y secuenciales. No pasamos a la sub-tarea B hasta que la sub-tarea A esté perfectamente resuelta y verificada.

Por ejemplo, para construir el sistema del coliseo del Barrio 27 de Mayo, dividimos el desarrollo en la siguiente cadena de tareas:

1. **Paso 1:** Diseñar únicamente la estructura de las tablas de la base de datos (usuarios, canchas, horarios).
2. **Paso 2:** Generar el código de conexión a la base de datos controlando errores de acceso.
3. **Paso 3:** Crear la interfaz gráfica o los endpoints para registrar una reserva.
4. **Paso 4:** Implementar las reglas de validación (que nadie pueda reservar la misma cancha a la misma hora).

A continuación se muestra un ejemplo de código en Python correspondiente al **Paso 2** de la cadena, enfocado exclusivamente en validar la conexión a la base de datos local sin mezclar otras funciones:



Python

```
# Importamos la librería encargada de conectar Python con el servidor
PostgreSQLimport psycopg2
# Importamos la librería que maneja errores específicos de la base de datosfrom
psycopg2 import OperationalErrordef conectar_servidor_local():    try:        #
Iniciamos la conexión con los parámetros específicos de nuestra red local en el
CEA        conexion = psycopg2.connect(            host="192.168.1.50",            #
Dirección IP fija asignada al servidor del coliseo            database="
db_coliseo",        # Nombre de la base de datos que almacena las reservas
user="
tecnico_pailon",        # Usuario técnico configurado con permisos limitados
password="
ClaveSegura123", # Contraseña robusta definida para el acceso local
port="5432"        # Puerto de red estándar para el servicio PostgreSQL
)        # Si la conexión no lanza ninguna excepción, mostramos un mensaje de
éxito en la consola        print("
Conexión exitosa al servidor del Barrio 27 de Mayo.")        # Retornamos el
objeto de conexión para ser usado en las siguientes sub-tareas        return
conexion    except OperationalError as error_tecnico:        # Si hay un fallo
(cable desconectado, servidor apagado, etc.), capturamos el error
print(f"
Error de conectividad en Pailón: {
error_tecnico}
")        # Retornamos None para indicar que la conexión no se pudo establecer
de forma segura        return None
```

4.3. VERIFICACIÓN CRUZADA (CROSS-CHECKING) DE LA INFORMACIÓN GENERADA

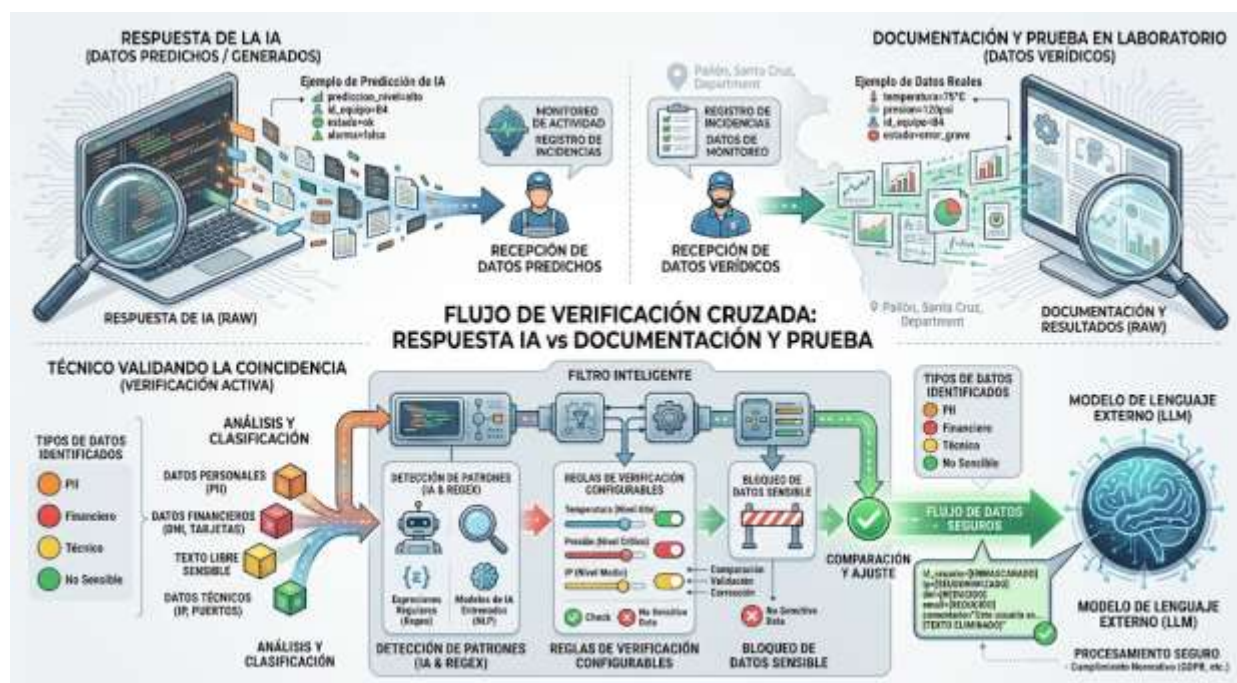
Las Inteligencias Artificiales no piensan como los humanos; predicen palabras basándose en patrones estadísticos. Esto significa que pueden sufrir "alucinaciones", inventando parámetros de configuración, librerías que no existen o rutas de archivos erróneas. El **Cross-checking** o verificación cruzada es la práctica obligatoria de contrastar cada dato proporcionado por la IA con fuentes de verdad oficiales y estables.

Para un Técnico que está instalando una red inalámbrica comunitaria en los barrios alejados como **San Expedito** o **Las Misiones**, aplicar verificación cruzada implica:

1. Confrontar los comandos de terminal que sugiere la IA con las páginas de ayuda oficiales de Linux (*man pages*) o la documentación oficial del fabricante del router.



2. Comprobar si las librerías de software que la IA propone descargar están vigentes en repositorios oficiales como PyPI o npm, verificando que no tengan vulnerabilidades de seguridad conocidas.
3. Validar las direcciones IP y máscaras de subred sugeridas mediante herramientas de cálculo de redes (*ipcalc*) antes de aplicarlas a los equipos de producción del cliente.



4.4. EVALUACIÓN CRÍTICA DE LA PRECISIÓN TÉCNICA DE LA RESPUESTA

Un Técnico no es un simple copiador de código; es un supervisor técnico. Evaluar críticamente significa examinar la respuesta de la IA bajo criterios de eficiencia, seguridad, costo y viabilidad local.

Cuando la IA genera una solución para automatizar el bombeo de agua o el control de iluminación en el módulo educativo propio del CEA en el **Barrio Saó**, debemos analizar la respuesta respondiendo las siguientes preguntas:

1. **Seguridad:** ¿La IA está dejando puertos abiertos innecesarios o incluyendo usuarios administradores sin contraseña?
2. **Optimización de recursos:** ¿El script consume demasiada memoria RAM o procesamiento para las computadoras limitadas con las que contamos en las salas de cómputo locales?



3. **Contexto geográfico y de infraestructura:** ¿La solución requiere internet satelital constante de alta velocidad o puede funcionar de manera offline en un servidor local dentro de Pailón?

Si la IA te sugiere un comando que borra directorios de forma recursiva sin pedir confirmación (rm -rf /), la evaluación crítica te permite detectar ese peligro antes de presionar *Enter* y destruir el servidor de archivos del CEA.

4.5. CREACIÓN DE UNA BIBLIOTECA PERSONAL DE PROMPTS EFECTIVOS

A medida que avanzas en tu carrera como Técnico, notarás que muchos problemas de soporte técnico se repiten: configurar impresoras en red, limpiar virus de almacenamiento USB, optimizar el arranque de Windows o restaurar respaldos de bases de datos en comercios del **Barrio María Belén** o **Barrio San Antonio**.

Guardar los prompts que te dieron resultados perfectos después de varias iteraciones te ahorrará horas de trabajo en el futuro. Una **biblioteca personal de prompts** es un archivo de texto ordenado, un repositorio en GitHub o una base de datos local donde almacenas tus plantillas de instrucciones estructuradas.

Un prompt bien estructurado para tu biblioteca debe incluir cuatro componentes esenciales:

Plaintext

```
[ROL]: Actúa como un Técnico en Sistemas Informáticos experto en soporte técnico de servidores Linux y redes locales.[CONTEXTO]: Estoy reparando el servidor de archivos de una escuela pública en Pailón que tiene cortes de energía constantes. El sistema de archivos EXT4 quedó marcado con errores tras un apagón masivo.[TAREA]: Proporcióname los comandos secuenciales usando la herramienta 'fsck' para escanear y reparar de forma segura la partición '/dev/sdb1' sin perder los datos de los estudiantes.[RESTRICCIÓN]: Incluye explicaciones breves en español para cada comando y detalla qué banderas usar para que repare los errores automáticamente de forma segura.
```



4.6. INGENIERÍA DE PROMPTS AVANZADA APLICADA AL DIAGNÓSTICO DE HARDWARE Y CONECTIVIDAD RURAL

En zonas rurales o poblaciones en crecimiento donde el acceso a internet técnico especializado es intermitente, las técnicas avanzadas como el *Few-shot Prompting* (dar ejemplos a la IA antes de pedirle el resultado) se vuelven herramientas críticas de diagnóstico para el Técnico.

Si estás reparando una placa madre o diagnosticando fallas en fuentes de poder de computadoras que llegaron al taller desde el barrio **Oto Waver** o **Ovidio Barberi**, puedes entrenar temporalmente a la IA en tu sesión de chat dándole dos ejemplos de fallas típicas de la zona (por ejemplo, sulfatación por humedad o picos de voltaje provocados por tormentas eléctricas) junto con sus soluciones. Al pedirle el diagnóstico del tercer equipo, la IA no te dará una respuesta genérica de fábrica de primer mundo, sino que priorizará las fallas típicas ocasionadas por las condiciones climáticas y eléctricas de nuestra región chiquitana.

4.7. AUTOMATIZACIÓN DEL REFINAMIENTO MEDIANTE METAPROMPTING

El **Metaprompting** consiste en utilizar la Inteligencia Artificial para que diseñe, evalúe o corrija tus propios prompts. Es "usar la IA para mejorar cómo usamos la IA". Como Técnico, muchas



veces sabes cuál es el problema, pero no sabes con qué términos técnicos exactos explicárselo al modelo informático.

Puedes usar una estructura de metaprompt como la siguiente: *"Quiero que configures un servidor web local Apache para el Barrio Jardines de Pailón, pero no sé qué parámetros pedirte para asegurar que funcione rápido en computadoras antiguas con solo 2 GB de memoria RAM. Hazme preguntas detalladas sobre mi entorno para que luego tú mismo construyas el prompt perfecto que debo ejecutar"*. La IA asumirá el rol de un consultor senior, te entrevistará y generará una instrucción técnica de alta precisión que nunca habrías podido redactar a la primera.

4.8. EL ROL DEL TÉCNICO COMO SUPERVISOR DE CALIDAD EN LA ERA DE LA INTELIGENCIA ARTIFICIAL

Hoy en día, las Inteligencias Artificiales escriben código a gran velocidad, pero carecen de sentido común y de responsabilidad legal o ética. El mercado laboral actual ya no busca Técnicos que solo sepan escribir líneas de código memorizadas, sino **Técnicos capaces de auditar, validar e integrar sistemas asistidos por IA**.

Si un sistema automatizado de control de asistencia falla en el **Barrio San José** o en **Las Misiones** debido a un bucle infinito generado por una IA mal supervisada, la responsabilidad no es de la IA, es del Técnico que copió y pegó la solución sin probarla. Tu valor profesional radica en tu capacidad para refinar la interacción con la tecnología, asegurando que cada línea de configuración sea segura, ética, estable y perfectamente adaptada al contexto donde se va a implementar.

CIERRE TEÓRICO OBLIGATORIO

❑ Mitos vs. Realidades

Mito	Realidad
Si el código generado por la Inteligencia Artificial no funciona a la primera, significa que la IA no sirve para resolver ese problema técnico.	Las IA requieren iteración y contexto específico. Un prompt inicial genérico produce resultados genéricos; el refinamiento progresivo del Técnico es lo que genera la solución exacta.



Es totalmente seguro copiar los scripts de configuración de red que da la IA directamente en los servidores de producción de los clientes en Pailón.

Es sumamente peligroso. Las respuestas de la IA pueden contener alucinaciones, comandos obsoletos o fallas de seguridad que deben pasar por verificación cruzada y entornos de prueba.

□ Glosario de Términos

1. **Iteración:** Proceso repetitivo donde se evalúa el resultado de una acción previa para introducir mejoras consecutivas en la siguiente instrucción.
2. **Encadenamiento (Chaining):** Técnica de diseño de prompts que consiste en fragmentar un problema complejo en una secuencia ordenada de sub-tareas más sencillas y controlables.
3. **Alucinación:** Fenómeno por el cual un modelo de Inteligencia Artificial genera información que suena coherente y verídica, pero que es técnicamente falsa, inventada o inexistente.
4. **Verificación Cruzada (Cross-checking):** Método de control de calidad informática que consiste en comparar los datos entregados por un software o IA con manuales técnicos, libros de texto oficiales o pruebas empíricas de laboratorio.
5. **Metaprompting:** Práctica avanzada que utiliza a la propia Inteligencia Artificial para crear, refinar o estructurar instrucciones óptimas dirigidas a otros modelos o a sí misma.

□ Ponte a Prueba

1. Cuando un Técnico en Sistemas Informáticos desglosa la creación de una base de datos grande para un comercio del Barrio Central Fátima en cuatro prompts pequeños e independientes, ¿qué técnica está aplicando?
 1. A) Alucinación controlada.
 2. B) Encadenamiento de prompts (Prompt Chaining).
 3. C) Metaprompting avanzado.
2. ¿Qué acción debe realizar un Técnico INMEDIATAMENTE después de que una IA le proporciona un script para eliminar virus en una computadora de la zona de Residencial Norte?



1. A) Ejecutar el script directamente con permisos de administrador.
 2. B) Guardar el script en la biblioteca de prompts sin hacer modificaciones.
 3. C) Evaluar críticamente el código y realizar una verificación cruzada con documentación oficial.
-
3. ¿Cuál es la función principal de incluir el "Contexto" dentro de una plantilla de nuestra biblioteca de prompts de soporte técnico?
-
1. A) Hacer que el prompt sea más largo para que la IA tarde más en responder.
 2. B) Proporcionar detalles específicos sobre el hardware, sistema operativo y limitaciones locales para que la respuesta sea precisa y aplicable.
 3. C) Evitar por completo que el Técnico tenga que volver a estudiar redes o hardware.



VALORACIÓN



Reflexión social, ética y ambiental en nuestra región: El uso de herramientas de Inteligencia Artificial para agilizar el trabajo técnico en Pailón trae consigo grandes ventajas, pero también serias responsabilidades éticas. Cuando un Técnico automatiza configuraciones sin realizar una evaluación crítica de las respuestas, puede provocar fallas informáticas graves en los sistemas de cobro de los pequeños comercios locales, afectando la economía familiar de los vecinos del Barrio Central Fátima o dañando equipos informáticos costosos por malas configuraciones de energía.

Desde el punto de vista medioambiental, cada consulta que realizamos a servidores de Inteligencia Artificial en la nube consume grandes cantidades de agua para la refrigeración de los centros de datos y una elevada cantidad de energía eléctrica. Si un Técnico no sabe iterar eficientemente y envía cientos de prompts genéricos por falta de orden, está contribuyendo innecesariamente al consumo energético global. Además, una mala instrucción de software que sobrecaliente los componentes de hardware de una computadora acelera el desgaste de los equipos, transformándolos prematuramente en basura electrónica (**e-waste**) que termina contaminando los suelos de nuestro municipio.



Por lo tanto, refinar nuestros prompts, trabajar con orden mediante sub-tareas y verificar minuciosamente la precisión del código no es solo una buena práctica profesional, sino un compromiso ético con la seguridad de la información de nuestros clientes y con el cuidado de los recursos de nuestra comunidad en el departamento de Santa Cruz.



PRODUCCIÓN



LABORATORIO PRÁCTICO: AUTOMATIZACIÓN DE RESPALDOS DE DATOS MEDIANTE PROMPTS ITERATIVOS Y ENCADENADOS

Objetivo: Crear un script robusto en Python que realice copias de seguridad de una base de datos local para una tienda en el **Barrio Residencial Norte**, simulando un entorno donde el Técnico aplica iteración, verificación cruzada y almacenamiento en su biblioteca personal.

Equipos necesarios: Computadora del laboratorio del CEA Herman Ortiz Camargo con Python 3 instalado y conexión a internet local.

Paso 1: El prompt inicial (Fallas comunes)

Abre tu herramienta de IA de preferencia e introduce este prompt deficiente de forma consciente: *"Dame un código en Python para hacer copia de seguridad de una carpeta"*.

Analiza críticamente el resultado. Notarás que el script que te entrega es genérico, no controla si el disco está lleno, no maneja errores de red locales y guarda los archivos con nombres fijos, lo que causaría que el respaldo del día siguiente borre el respaldo del día anterior.

Paso 2: Aplicando la técnica de iteración y ajuste progresivo

Ahora, vas a refinar la respuesta de la IA. En el mismo chat, introduce el siguiente prompt adaptado a las necesidades de conectividad y almacenamiento seguro de los comercios de Pailón:



Plaintext

El script que me diste es muy básico. Modifica el código en **Python** bajo las siguientes condiciones: 1. Debe copiar todos los archivos de la carpeta ' C:\DatosTienda ' a una carpeta de respaldo llamada ' D:\RespalDOSPailon '. 2. El archivo comprimido final (.zip) debe incluir en su nombre la fecha actual (Año-Mes-Día) para evitar que se sobrescriban los respaldos anteriores. 3. Si la carpeta de origen no existe, el programa debe mostrar un error claro en pantalla en lugar de colapsar. 4. Añade control de excepciones por si el almacenamiento de destino se queda sin espacio.

Paso 3: Análisis y verificación del código generado

La IA te entregará un código refinado. Tu tarea como Técnico es evaluar el script línea por línea antes de ejecutarlo. El código final estructurado y verificado de forma cruzada debe lucir y funcionar exactamente como se detalla a continuación:

Python

```
# Importamos La Librería '
os
' que nos permite interactuar con las carpetas del sistema operativoimport os
# Importamos La Librería '
shutil
' encargada de realizar operaciones de copiado y compresión de archivosimport
shutil
# Importamos La Librería '
datetime
' para obtener la fecha exacta del sistema de la computadorafrom datetime import
datetimedef ejecutar_respaldo_comercial():    # Definimos Las rutas absolutas de
origen y destino en el almacenamiento de la computadora    ruta_origen = r"
C:\DatosTienda"    ruta_destino_base = r"
D:\RespalDOSPailon"    # Verificación crítica: Comprobamos de forma segura
si la carpeta con los datos reales existe    if not os.path.exists(ruta_origen):
# Si no existe, imprimimos un aviso detallado para el usuario en el Barrio
Residencial Norte    print(f"
Error Técnico: La ruta de origen '{
ruta_origen}
' no fue encontrada. Verifique el disco duro.")    return # Finalizamos la
función de forma controlada para evitar que el programa falle por completo
# Si la carpeta de respaldos generales no existe en la unidad D, el script la
crea de forma automática    if not os.path.exists(ruta_destino_base):
os.makedirs(ruta_destino_base)    # Mostramos en la consola del sistema que
la carpeta fue creada con éxito    print(f"
```



```

Carpeta base de respaldos creada en: {
ruta_destino_base}
")      # Obtenemos la fecha actual del sistema formateada con año, mes y día
de forma ordenada      fecha_actual = datetime.now().strftime("%Y-%m-%d")      #
Construimos el nombre final del archivo comprimido adjuntando la fecha del día
del respaldo      nombre_archivo_respaldo = f"
Respaldo_{
fecha_actual}
"      # Unimos la ruta base del disco de destino con el nombre específico del
nuevo archivo comprimido      ruta_destino_final = os.path.join(ruta_destino_base,
nombre_archivo_respaldo)      try:      # Imprimimos en la pantalla de la
consola el inicio del proceso de compresión de archivos      print("
Iniciando la compresión de los datos del negocio local en Pailón...")      #
La función '
make_archive
' comprime la carpeta de origen en formato ZIP en la ruta de destino
shutil.make_archive(ruta_destino_final, '
zip', ruta_origen)      # Confirmamos al Técnico en la terminal que la
operación concluyó sin inconvenientes técnicos      print(f"
Respaldo completado exitosamente. Archivo guardado como: {
ruta_destino_final}
.zip")      except OSError as error_sistema:      # Capturamos fallos del
sistema como falta de espacio en disco o problemas de permisos de escritura
print(f"
Fallo crítico durante el respaldo en Pailón: {
error_sistema}
")      except Exception as error_inesperado:      # Capturamos cualquier otro
tipo de error imprevisto para mantener la estabilidad del software
print(f"
Ocurrió un error inesperado controlado por el Técnico: {
error_inesperado}
")
# Bloque estándar de ejecución que arranca nuestra función principal al ejecutar
el archivo de Pythonif __name__ == "__main__":      ejecutar_respaldo_comercial()

```

Paso 4: Construcción de tu entrada para la biblioteca de prompts

Una vez comprobado que el script funciona sin errores en la computadora del laboratorio, copia el prompt del **Paso 2** junto con los parámetros que añadiste y guárdalo en un archivo de texto llamado biblioteca_prompts_redes.txt en tu computadora. Este prompt ya forma parte de tu activo profesional como Técnico en Sistemas Informáticos.

RECURSOS ADICIONALES

1. **Documentación Oficial de Python (Módulo Shutil):** Manual técnico oficial en español para aprender cómo mover, copiar y comprimir archivos mediante código de forma segura,



ideal para realizar verificaciones cruzadas de scripts de soporte técnico. (Disponible en la web oficial de Python Docs).

2. **Guía de Ingeniería de Prompts de OpenAI:** Repositorio oficial de buenas prácticas y estrategias detalladas paso a paso para desglosar tareas complejas y reducir las alucinaciones en modelos de lenguaje grandes. (Disponible en el portal de desarrolladores de OpenAI).
3. **Repositorio de Guías Técnicas de Linux (GNU Coreutils):** Documentación de referencia para validar comandos críticos del sistema de archivos y administración de redes antes de aplicarlos en entornos reales de producción. (Disponible en la web oficial del proyecto GNU).